

Demo 展示讲解备注

内部讲稿 • 现场展示步骤、讲法提示与追问应对

2026 年 5 月 14 日

目录

0. 开场：展示目标	1
1. 要解决的问题	2
2. 系统架构	2
3. 现场运行步骤	2
4. TaskSpec	3
5. Agent 闭环	3
6. MuJoCo 执行	4
7. 严格评估	4
8. AutoRetry	4
9. Experiment Memory	5
10. 技术原理	5
11. 当前边界	5
12. 下一步演进	6
13. 结尾	6

这份是自己看的讲稿，不给面试官看。

面试官看的正式版本是 Demo 展示介绍 _10 分钟.md。

0. 开场：展示目标

屏幕停留

打开 Demo 展示介绍 _10 分钟.md 的“展示目标”部分。

建议讲法

我这个 Demo 的核心不是展示一个固定机械臂动作，而是展示一个实验工作流 Agent。用户给一个自然语言实验目标，系统会把它转成结构化协议，然后调用 MuJoCo 作为虚拟实验仪器执行，回收物理指标和失败类型，再生成重试计划、自动跑一轮修正，并写入实验记忆。

要强调

- 不是机器人动画。

- 不是端到端控制器。
- 是实验闭环 Agent。

可能追问

如果问“这算 Agent 吗”，回答：算实验 workflow Agent，不是端到端机械臂控制 Agent。

1. 要解决的问题

屏幕停留

“要解决的问题”部分。

建议讲法

以“推动方块移动 5cm，不能陷进桌子”为例，它看起来是一句话，但真正执行时系统要知道任务类型、机器人、对象、目标位移、物理安全约束、成功判定和失败后的修正方向。所以关键是把自然语言转成可执行协议。

要强调

- 5cm 不是文本，而是目标位移。
 - 不能陷进桌子不是描述，而是安全约束。
 - 失败后要能归因，不是只返回失败。
-

2. 系统架构

屏幕停留

架构图部分。

建议讲法

系统分成三层：上层 Agent Orchestrator 做目标理解、计划和归因；中间协议层用 TaskSpec、ExperimentPlan、SkillPlan 固定任务；底层 MuJoCo、动作原语和 IK 执行；最后 EvaluationReport、RetryPlan、ExperimentStore 负责反馈和记忆。

要强调

Agent 不直接控制关节。这样更安全，也方便验证。

可能追问

如果问“这和 pipeline 有什么区别”，回答：pipeline 是执行骨架，Agent 体现在任务解析、实验设计、失败归因和 retry 决策这些动态节点。

3. 现场运行步骤

屏幕操作

打开 PowerShell，执行：

```
cd D:\HuaweiMoveData\Users\hw\Desktop\try
powershell -ExecutionPolicy Bypass -File .\scripts\open_mujoco_web_demo.ps1
```

如果已经启动了，就打开：

`http://127.0.0.1:8765/health`

确认返回：

```
{"ok": true, "mode": "mujoco"}
```

网页输入

用 fr3 机械臂推动方块移动 5cm，不能陷进桌子

建议讲法

这条输入能同时展示自然语言理解、目标位移、物理安全、严格成功率、自动重试和实验记忆。

4. TaskSpec

屏幕操作

网页运行后，先看任务解析/TaskSpec 区域。

建议讲法

这里可以看到系统识别出任务是 push，机器人是 franka_fr3，对象是 cube_5cm。最重要的是 5cm 被写成 `target_displacement_m = 0.05`，不能陷进桌子被写进 table clearance 相关约束。

要强调

TaskSpec 是自然语言到可执行实验的桥。

可能追问

如果问“LLM 输出错怎么办”，回答：不会直接执行裸输出，后端还有对象、动作、机器人和成功标准的校验与后处理。

5. Agent 闭环

屏幕操作

看页面里的闭环条：

```
TaskSpec / ExperimentPlan / SkillPlan / Capability / MuJoCo / Evaluation / RetryPlan / AutoRetry / Memory
```

建议讲法

这条链就是 Demo 中 Agent 的工作边界。它先把任务协议化，再生成实验计划和技能计划，然后调用 MuJoCo 执行。执行完成后，系统读取结果，生成评估报告，再根据失败类型生成重试计划并自动执行一轮修正。

要强调

现在已经不只是 RetryPlan 建议，而是会执行 AutoRetry。

6. MuJoCo 执行

屏幕操作

看轨迹回放和 runs 日志。

建议讲法

MuJoCo 在这里是虚拟实验仪器。系统不是只播放动画，而是执行真实物理仿真，记录物体位移、接触步数、最大接触力、末端位置和回放帧。这些数据会进入后续评估。

可能追问

如果问“这是真实机器人吗”，回答：当前是 MuJoCo 仿真，不是真实硬件；真实硬件通过 Adapter 层接入，必须先通过安全检查。

7. 严格评估

屏幕操作

看 EvaluationPanel、runs 表格里的：

- target_displacement_m
- object_displacement
- displacement_error_m
- table_clearance_ok
- failure_type

建议讲法

之前很容易出现一个问题：只要物体动了就算成功。现在推动任务的成功条件是实际位移必须在目标 5cm 的容差内，同时桌面安全必须满足。所以移动 30cm 是 overshoot，没推够是 undershoot，碰桌或穿桌是 table_penetration。

要强调

成功率不是为了好看，而是严格绑定目标和物理约束。

8. AutoRetry

屏幕操作

看自动闭环重试区域。

建议讲法

如果首轮实验有失败类型，系统会选择主导失败类型，生成 RetryPlan，再生成 revised task spec 自动跑一轮重试。例如 table_penetration 会提高接近高度、收紧 clearance，并调整推力空间，然后比较重试前后的成功率和原失败类型占比。

重要解释

如果重试后成功率没变，但 `table_penetration` 变成 `undershoot`，这不一定是坏事。说明系统已经把“物理不安全”修正成“推得不够”，失败问题更细了，下一轮可以继续增加推力或调整接触点。

9. Experiment Memory

屏幕操作

看实验记忆区域，必要时打开：

```
results/nlp_experiment_store.jsonl
```

建议讲法

每轮实验都会写入实验记忆，包括 `TaskSpec`、`ExperimentPlan`、`SkillPlan`、`EvaluationReport`、`RetryPlan` 和 `retry execution`。这样实验不是一次性结果，而是可以被后续检索、统计和复用。

可能追问

如果问“现在记忆怎么用”，回答：当前已经写入和汇总；下一步会让下一轮实验设计读取历史最佳参数和高频失败类型。

10. 技术原理

建议讲法

底层动作执行用动作原语和 Jacobian IK。IK 公式是阻尼最小二乘，适合基础 `reach`、`push`、`press`、`insert`。高层 Agent 不直接输出关节控制，而是输出结构化计划，底层再执行。

公式

```

$$dq = J^T (J J^T + \lambda^2 I)^{-1} e$$

```

要强调

这种分层设计比让 LLM 直接控制机械臂更安全、更可验证。

11. 当前边界

建议讲法

当前已经完成最小闭环：自然语言到协议、MuJoCo 执行、严格评估、失败归因、一次自动重试和实验记忆。但还不是最终通用系统。主要差距是长期多轮自主实验、完整全局运动规划、真实机械臂接入、大规模统计评估和历史经验复用。

要强调

主动讲边界会显得更专业，不要把 Demo 夸成完整系统。

12. 下一步演进

建议讲法

下一步不是推倒重做，而是沿着现有接口扩展：

- Retry 从一轮变多轮；
 - MotionPlan 接 MoveIt/OMPL/TrajOpt；
 - Adapter 接真实机械臂；
 - EvaluationReport 加统计置信度；
 - ExperimentStore 用于主动实验设计。
-

13. 结尾

建议讲法

这个 Demo 的核心价值是把实验目标变成协议，把协议变成可执行实验，把执行结果变成结构化反馈，把失败反馈变成下一轮行动，再把实验沉淀为记忆。

最后一句可以说：

当前系统已经证明一个实验工作流 Agent 的最小闭环是可行的。下一步是把一次自动重试扩展成长期多轮自主实验，并逐步接入更强运动规划和真实实验设备。